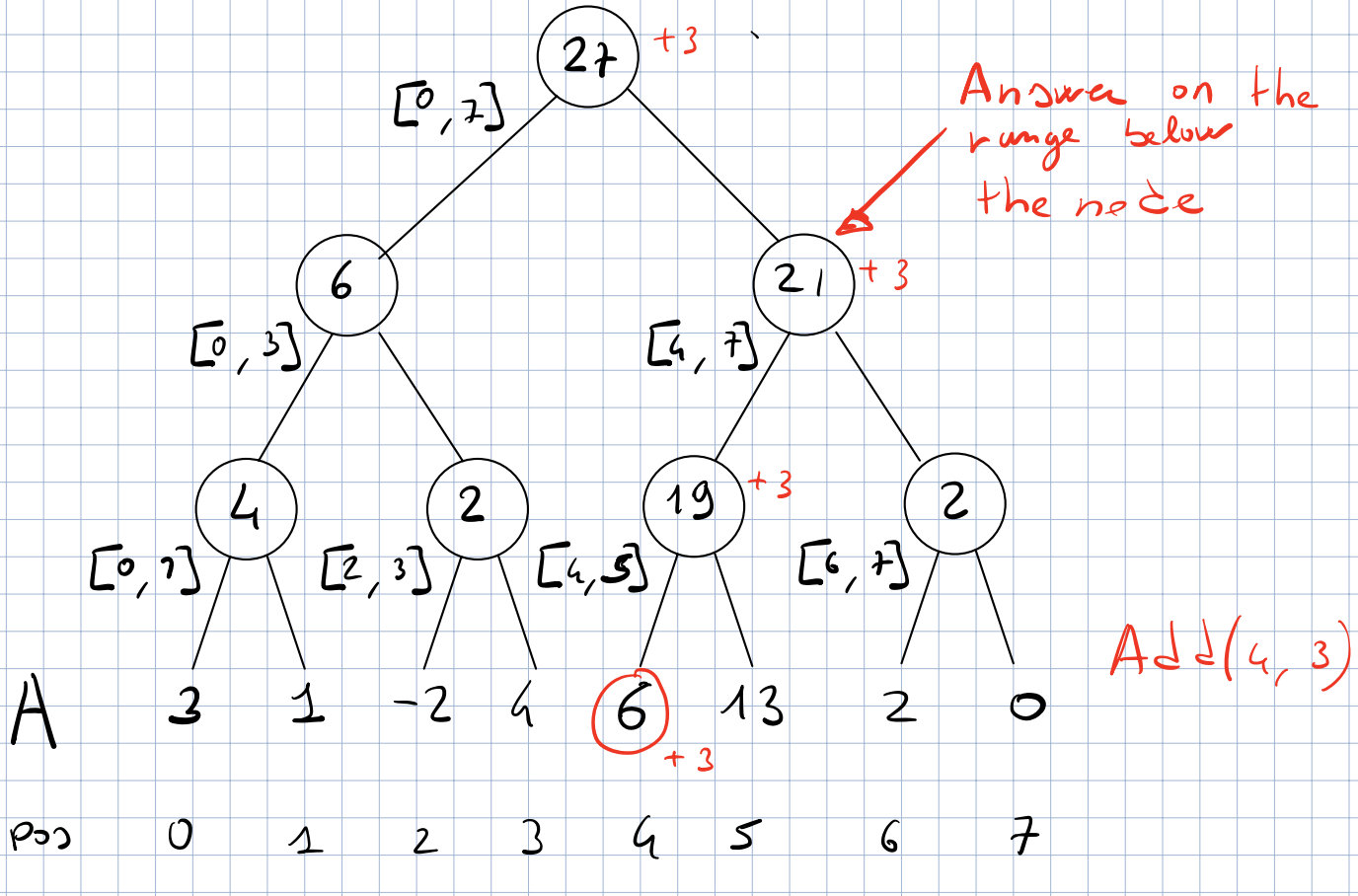
Segment Tree

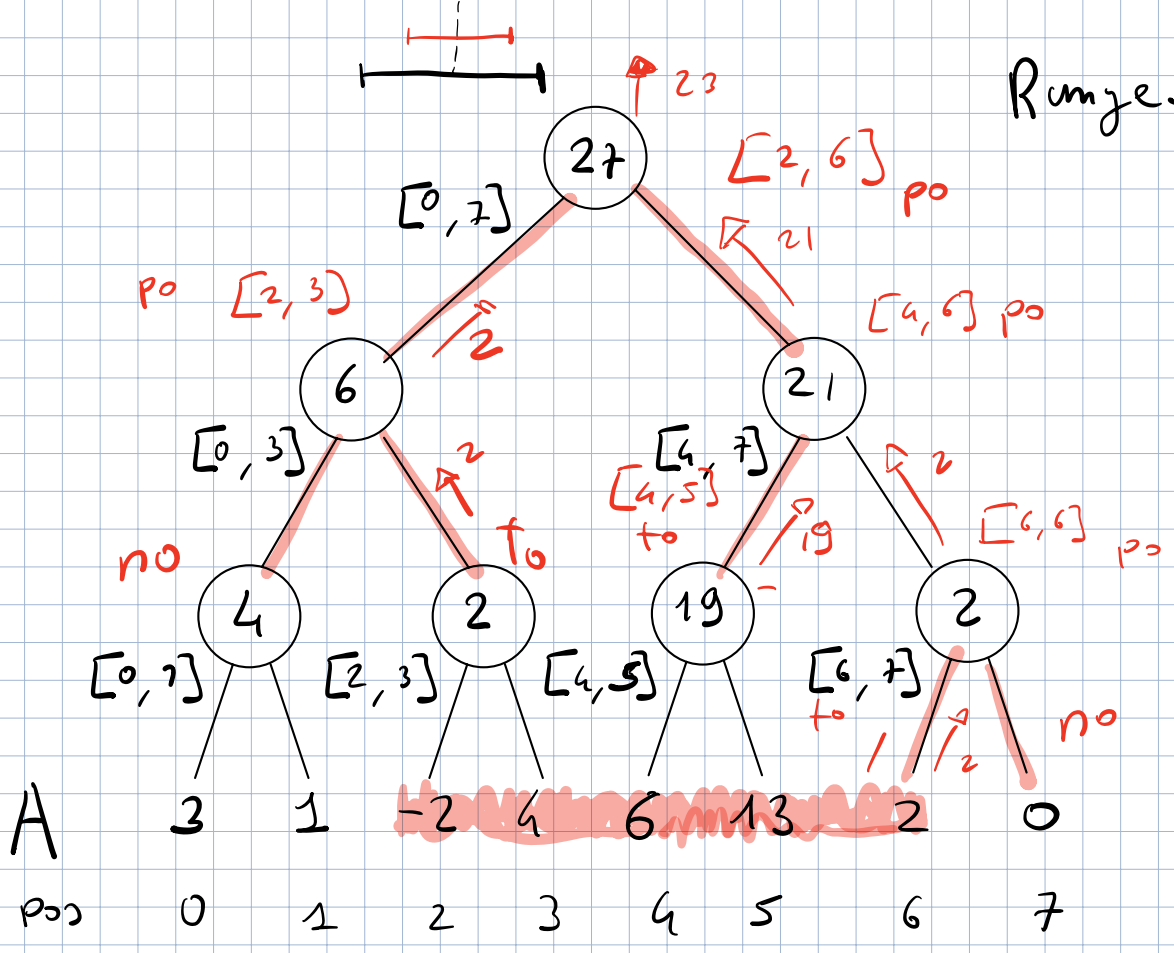
ST for Dynamic Prefix Sums

- Add (i, v) $\quad A[i] \mathrel{+}= v$
- Range Sum (i, j) $= \sum_{k=i}^{j} A[k]$



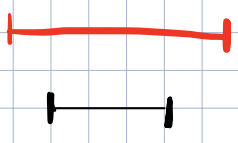
Update needs to modify $O(\log n)$ nodes on a root-to-a-leaf path

RangeSum(2, 6)



- Three cases

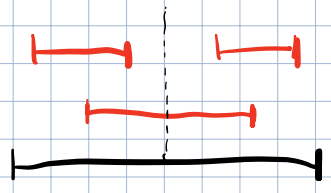
① Total overlap



query node

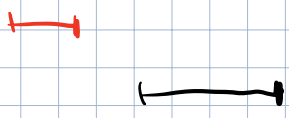
Stop recursion and get the answer

② Partial overlap



Recursively go both left and right

③ No overlap



Stop recursion

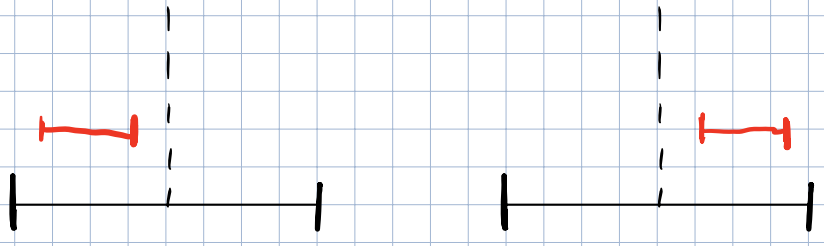
What's the time complexity of Range Sum?

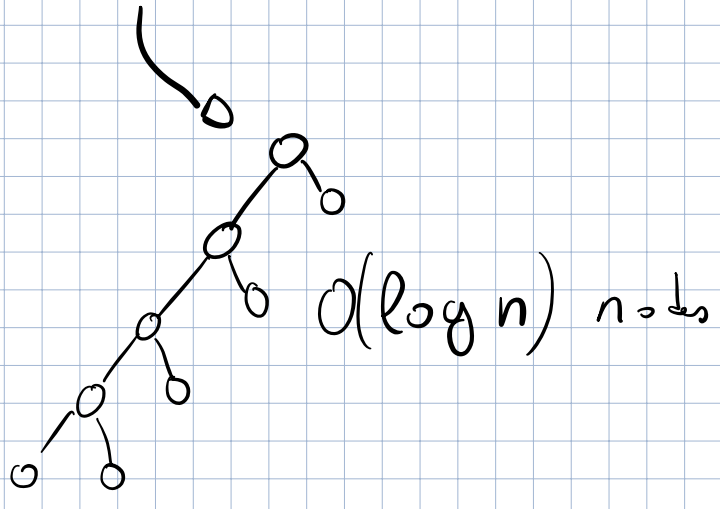
How many nodes do we process?

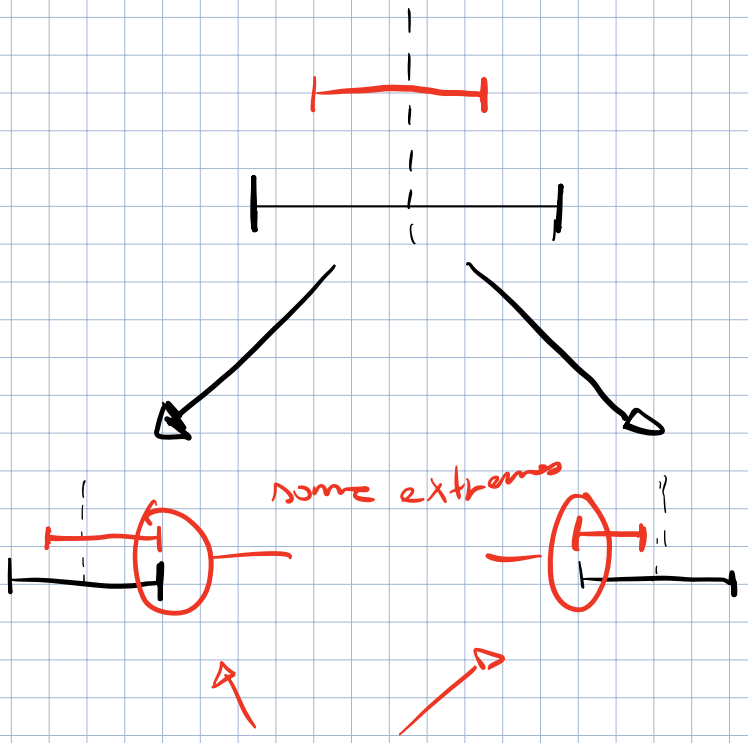
partial overlap



easy cases for partial overlap





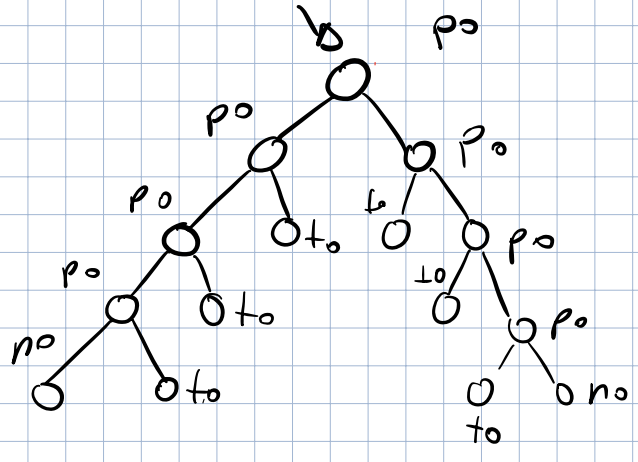


partial very cases

overlaps special

but and easy

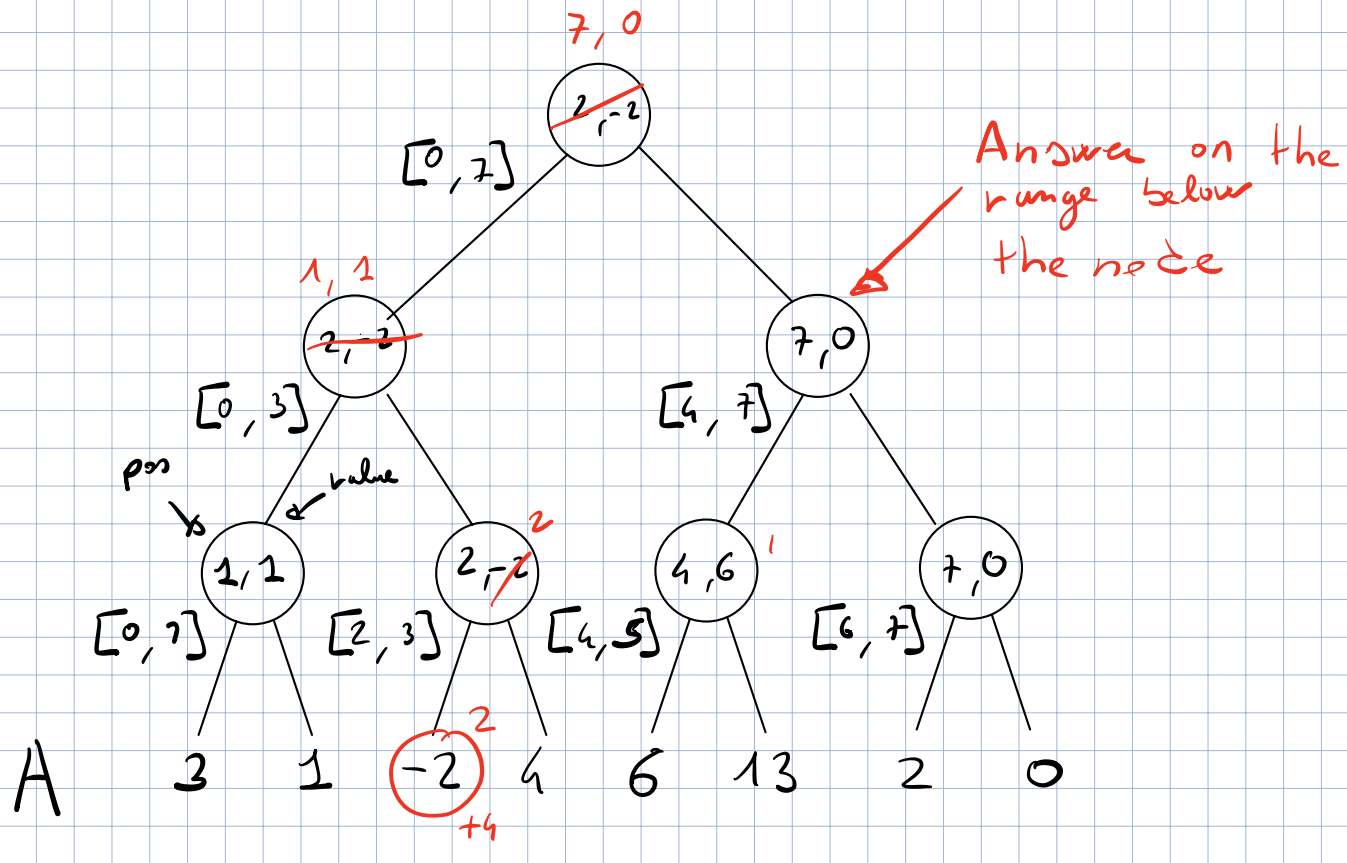
the only general po


po
po
po

Dynamic Range Minimum Queries

- Add (i, v)
- RMQ (i, j) = $\arg\min$ (A[i .. j])

↑ position of min

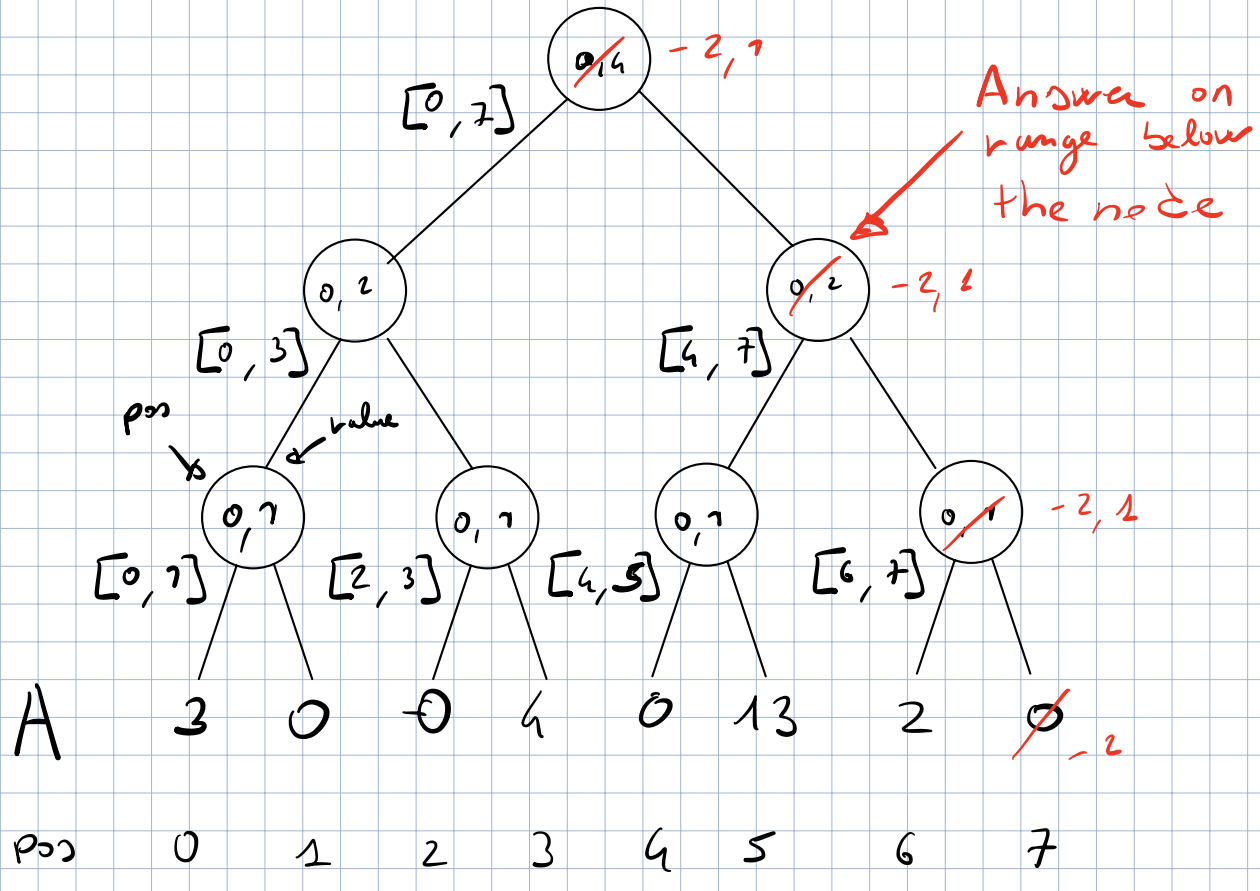


Add (2, 4)

RMQ with occurrences

- Add (i, v)

- query (i, j) = report the minimum value in the range in A[i ... j] with its # of occurrences in A[i ... j]



m, c m', c'

if m == m' c + c'

Problem

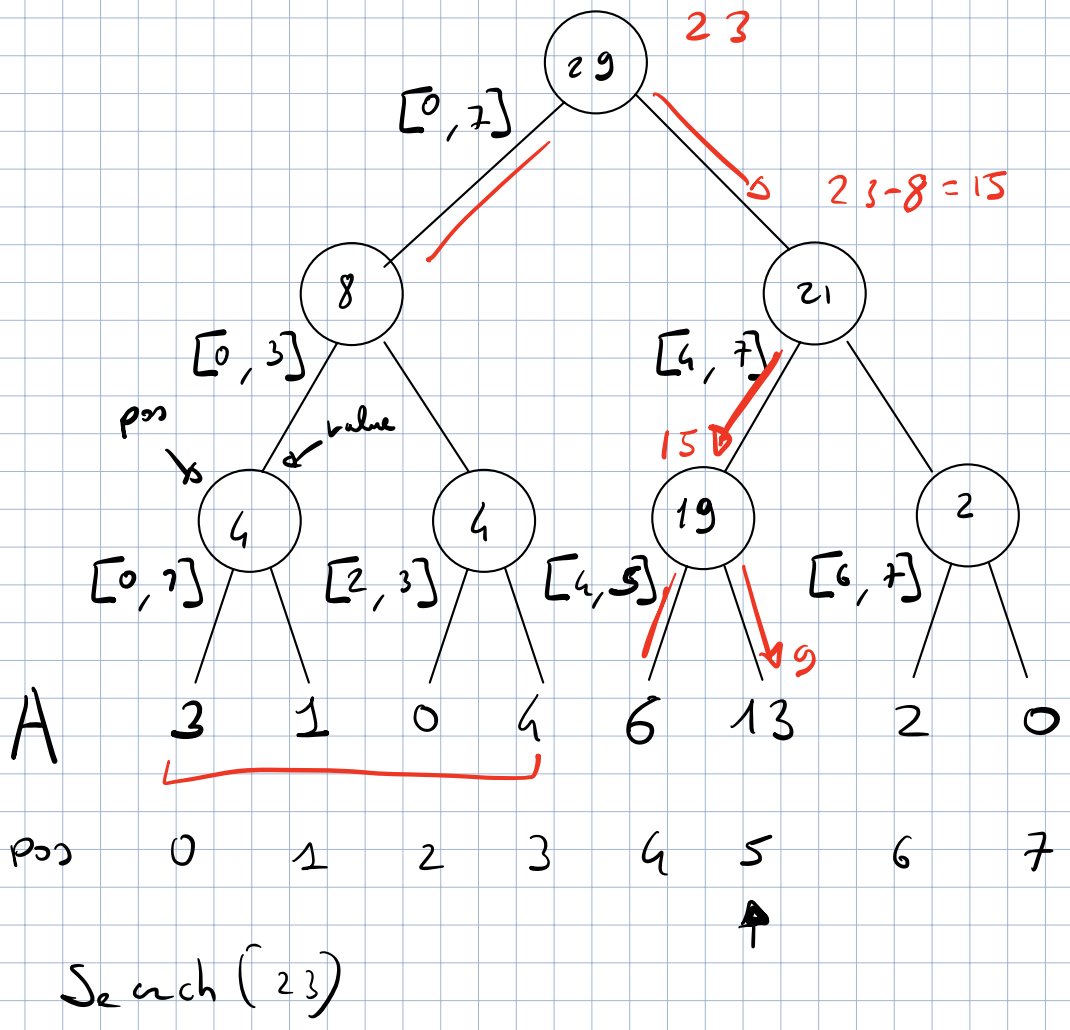
$A[1, n]$ with non-negative integers

- Add(i, v)
- RangeSum(i, j)
- Search(s) : reports the position i of the shortest prefix of A such that

$$\sum_{k=1}^{i} A[k] \geq s$$

A	3	1	0	4	6	13	2	0
	0	1	2	3	4	5	6	7

Search(22)



Naive Solution: Binary Search for the answer

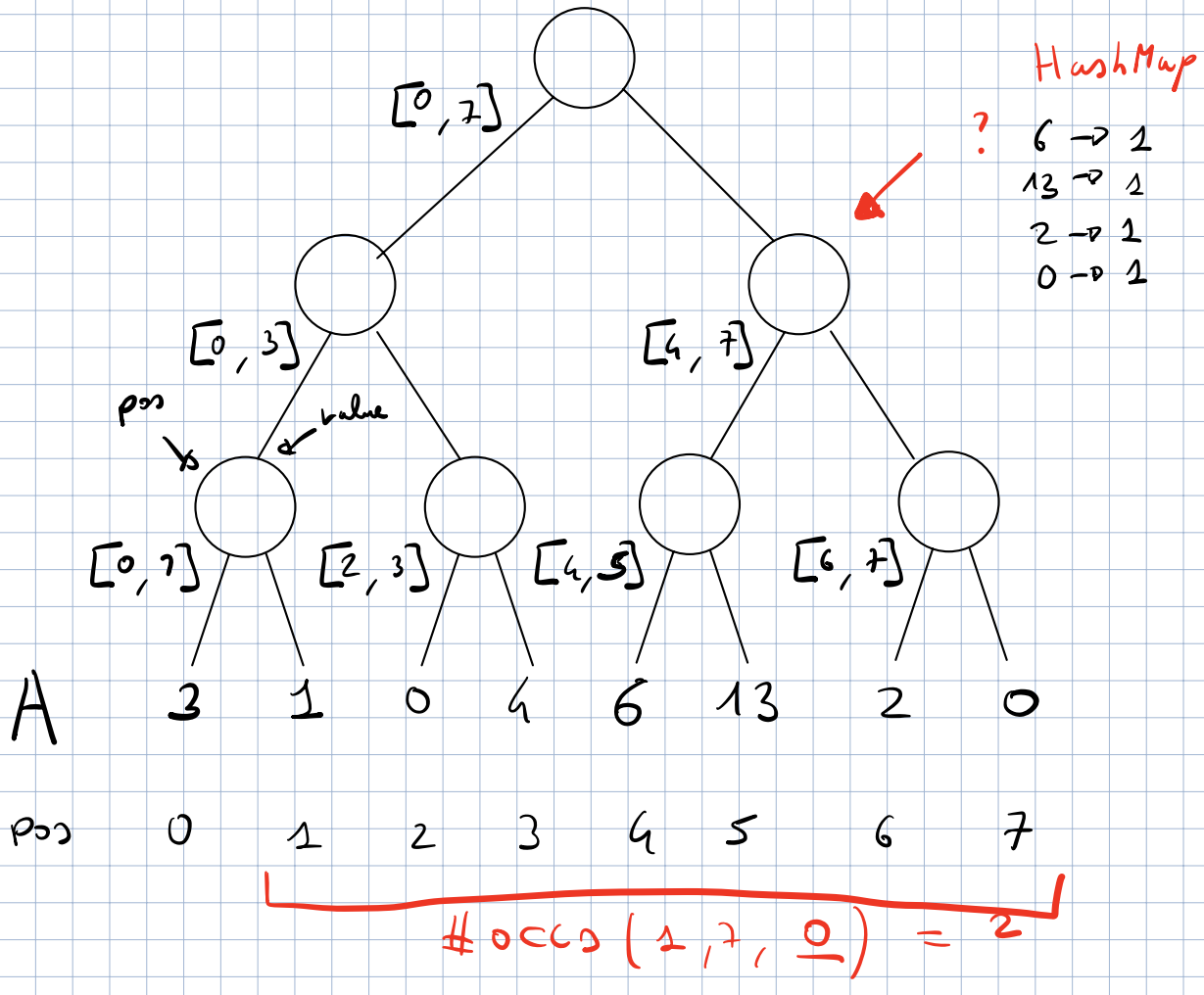
$\Theta(\log^2 n)$ time

Specialized Solution runs in $\Theta(\log n)$ time

Problem

Given A of integers

- Add(i, v)
- #occs(i, j, x) = reports the # of occurrences of x in A[i, j]

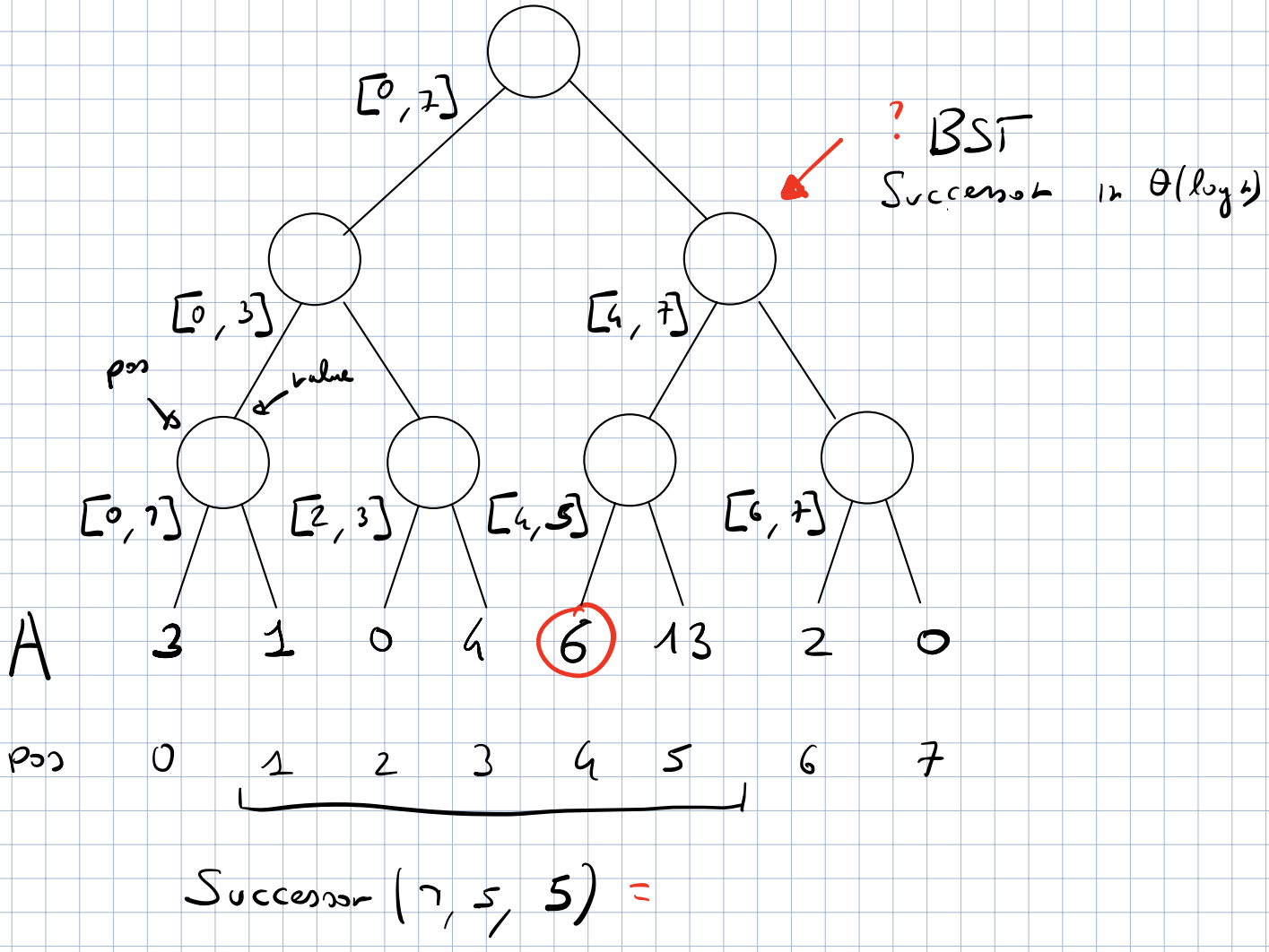


query time is $\Theta(\log n)$ (whp)

Space Usage is $O(n \log n)$ (more space efficient solution Wavelet trees)

Problem

- Add (i, v)
- Successor (i, j, x) report the successor of x in $A[i, j]$



Successor $(7, 5, 5) =$

- Compute the successor in every total overlapping node
- Combine by taking the min

- Query time $\Theta(\log^2 n)$
- Space Usage is $\Theta(n \log n)$

we query time can be improved to $\Theta(\log n)$ time with fractional cascading

Triplets

Given $A[0, n-1]$ with integers

count the number of triplets

i, j, k $(0 \leq i < j < k < n)$

such that

$$A[i] < A[j] < A[k]$$

Example

A 1 2 3 4 1

result = 4

1 2 3
1 2 4
2 3 4
1 3 4

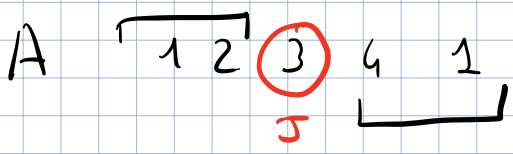
Solution in $\Theta(n^3)$: check every possible triplets

Solution in $\Theta(n^2)$

A 1 2 (3) 4 1
j

How many triplets with that j ?

k = # of $A[i] < A[j]$

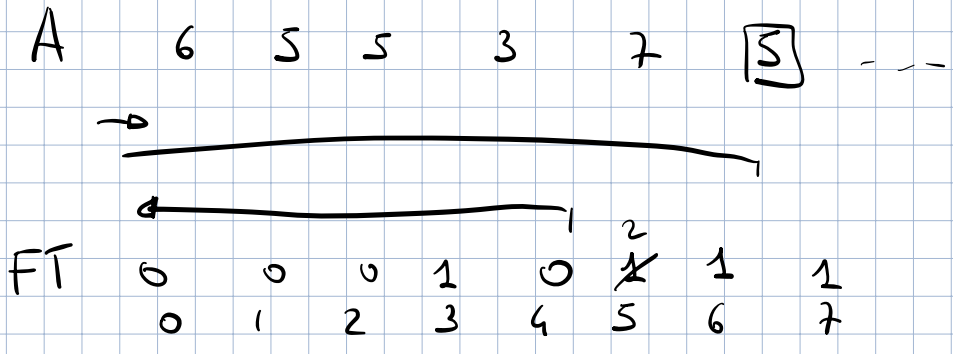


k' = # of $A[j] < A[k]$

$k = 2$ $\quad$ $k' = 7$

$k \cdot k' = 2$

Solution in $\Theta(n \log n)$ time



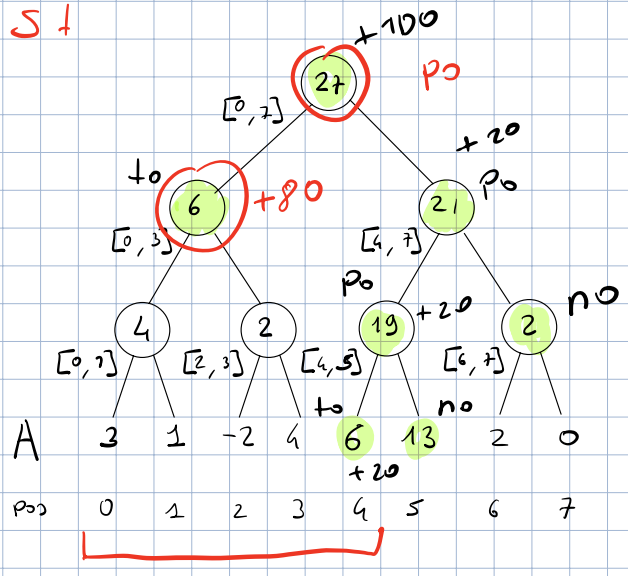
Range Update with Lazy Propagation

Given A[1, n]

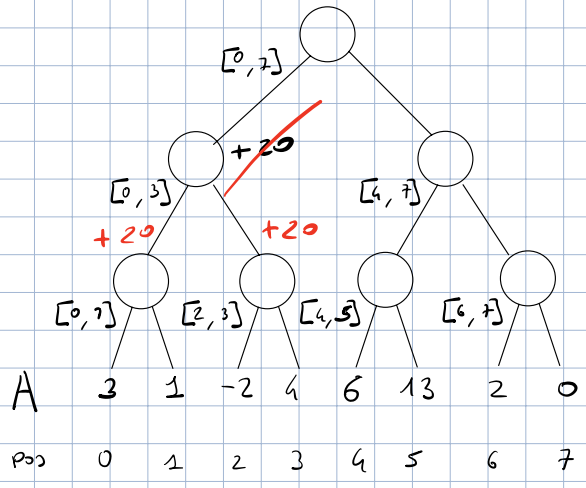
- Range Update (i, j, v)
- Sum (i)

Solution: Lazy propagation

ST



Lazy tree



Range Update (0, 4, 20)

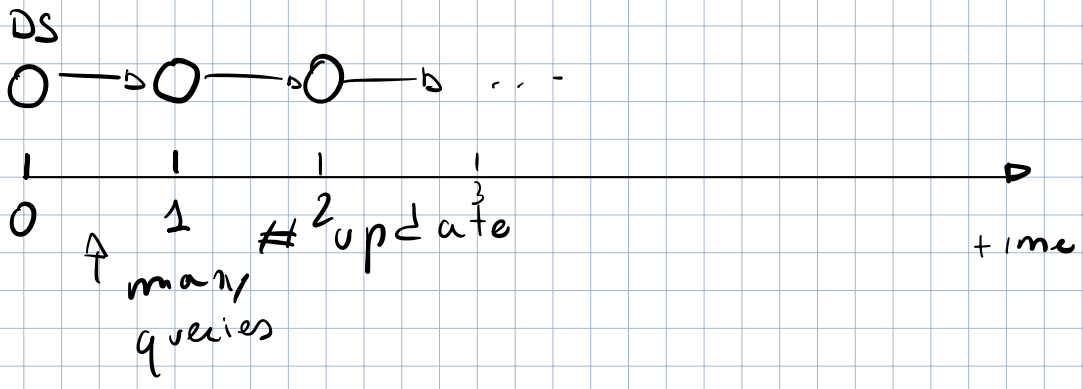
- Every time po, apply the change to the node in ST
- If to, remember the update in the Lazy tree. Unless it's a leaf.
- If no, do nothing

What's the running time? $\Theta(\log n)$ time

$Sum(3) = RangeSum(0, 3)$

runs in $\Theta(\log n)$ time

Persistent Segment tree



- ops that change the DS, move the timestamp by 1

- ops that query the DS, are now allowed to specify a valid timestamp

e.g. RangeSum(i, j, t)

$= \sum_{k=i}^{j} A[k]$ at time t

Persistent Range Sum

Given $A[0, n-1]$

- Add (i, v)
- RangeSum(i, j, t)

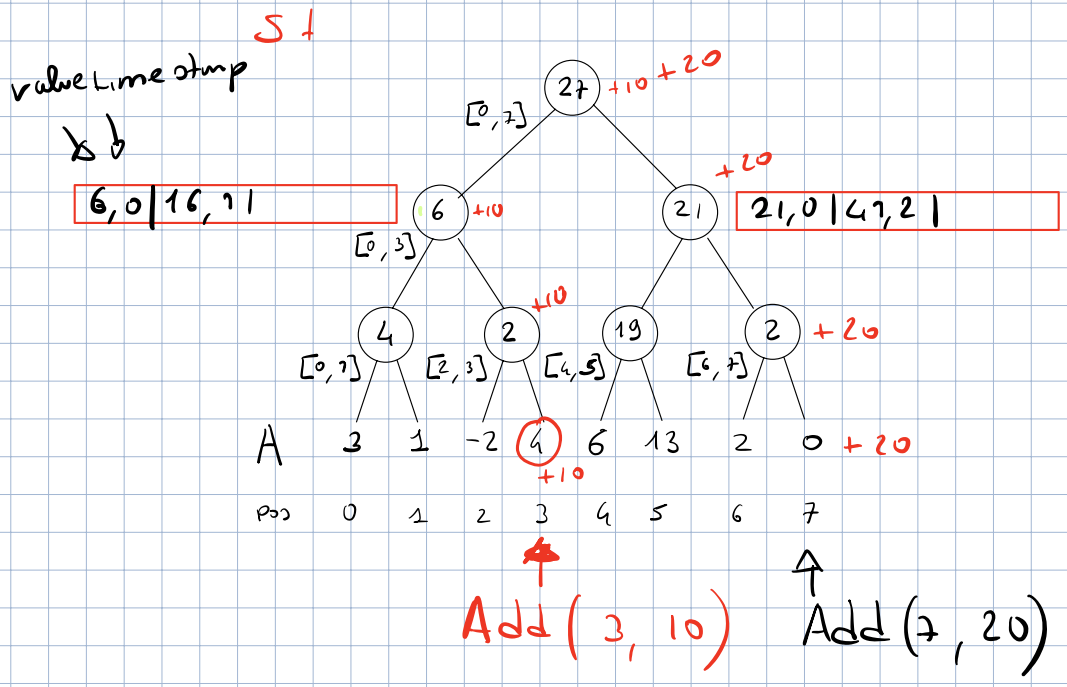
Solution(s)

Attempt # 1

- Make a copy of ST after each update
  - Pro: RangeSum takes $\Theta(\log n)$ time
  - Cons: - Space Usage is $\Theta(n \cdot t)$ after t updates
    - Add operation takes $\Theta(n)$ time

Attempt # 2

Exploit: we have just $\Theta(\log n)$ changes in the ST after every update!



Pro: - Add takes $\Theta(\log n)$ time

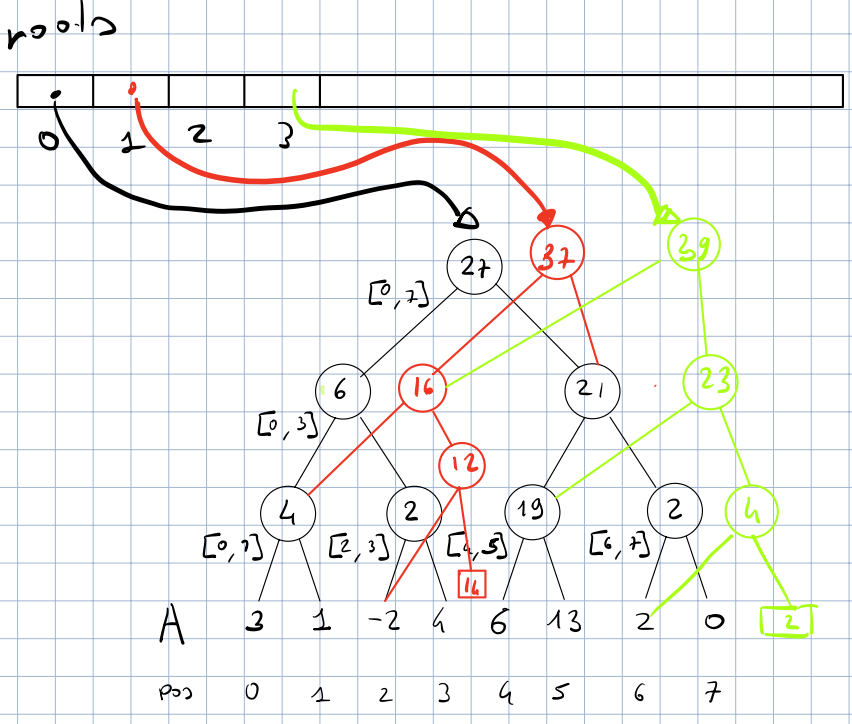
- Space $\Theta(n + t \log n)$

RangeSum(i, j, t)

- Search for predecessor of t in every list of total overlapping nodes
- time complexity: $\Theta(\log n \cdot \log t)$

Attempt # 3

Exploit: the $\Theta(\log n)$ are on a root-to-a-leaf path



- RangeSum takes $\Theta(\log n)$ time
- Add takes $\Theta(\log n)$ time
- Space Usage: $\Theta(n + t \log n)$ after t updates